

Explanation of `diego_analysis.R`: Cardiovascular Disease Logistic Regression Analysis

STAT 4355 Project Analysis

2026-05-12

Contents

1 Purpose of the File	2
2 Packages and Data Loading	2
3 Commented-Out Preprocessing	3
4 The First Logistic Regression Model: <code>cardiomodel</code>	3
5 The Reduced Main-Effects Model: <code>cardiomodel2</code>	4
6 Comparing <code>cardiomodel</code> and <code>cardiomodel2</code>	4
6.1 AIC	5
6.2 Likelihood Ratio Test	5
6.3 Single-Term Drop Tests	5
7 DHARMA Residual Diagnostics for <code>cardiomodel2</code>	5
8 Train-Test Prediction Using <code>cardiomodel2</code>	6
9 Classification Metrics	7
10 Density Plot of Predicted Probabilities	7
11 Threshold Tuning	8
12 Attempted Full Interaction Model: <code>cardiomodel3</code>	8
13 Forward Selection Attempts: <code>cardiomodel6</code>, <code>cardiomodel6b</code>, and <code>cardiomodel6c</code>	8
13.1 Interaction-Block Forward Selection	9
13.2 Main-Effect Forward Selection	9
13.3 Selected Candidate Forward Selection	9
13.4 Important Coding Note	9

14 LASSO Logistic Regression with Pairwise Interactions	10
15 Selected Terms and the Hierarchy Principle	10
16 Post-LASSO Logistic Regression: <code>cardiomodel8</code>	11
17 Pruning the Post-LASSO Model	11
18 Residual Diagnostics for the LASSO-Based Models	12
19 ROC Curve and AUC	12
20 Odds Ratio Plot for Presentation	13
21 AIC Comparison Plot	13
22 Important Interactions Identified	13
23 Overall Interpretation of the Script	14
24 Recommended Summary for a Presentation	15
25 Potential Improvements	15

1 Purpose of the File

The file `diego_analysis.R` performs a logistic regression analysis on a cleaned cardiovascular disease dataset. The main response variable is

$$cardio_i = \begin{cases} 1, & \text{if patient } i \text{ has cardiovascular disease,} \\ 0, & \text{if patient } i \text{ does not have cardiovascular disease.} \end{cases}$$

The goal of the script is to identify factors associated with cardiovascular disease risk and to compare several logistic regression models. The script also evaluates predictive performance, checks residual diagnostics, performs threshold selection, explores interaction terms, and creates plots suitable for presentation.

2 Packages and Data Loading

The script begins by installing and loading several R packages:

```
install.packages("ggplot2", type = "binary")
install.packages("DHARMa", type = "binary")
install.packages("MASS", type = "binary")
install.packages("olsrr", type = "binary")
install.packages("glmnet")
install.packages("pROC")
```

```
library(car)
library(ggplot2)
library(DHARMA)
library(MASS)
library(olsrr)
library(glmnet)
library(pROC)
```

The important packages are:

- `ggplot2`: creates plots.
- `DHARMA`: creates simulated residual diagnostics for generalized linear models.
- `glmnet`: fits LASSO logistic regression models.
- `pROC`: creates ROC curves and calculates AUC.
- `car`: can be used for variance inflation factors, although the VIF code is commented out.

The file then reads in the cleaned dataset:

```
readLines("filtered_cardio.csv", n = 5)
t <- read.csv2("C:\\Users\\dx230000\\Desktop\\filtered_cardio.csv")
names(t)
```

The command `readLines()` checks the first few lines of the file. The command `read.csv2()` loads the dataset. The function `read.csv2()` is used for files that are commonly separated by semicolons rather than commas. The object `t` stores the dataset.

3 Commented-Out Preprocessing

Several preprocessing steps are included but commented out. These include filtering blood pressure values, removing missing observations, removing unrealistic BMI values, converting height and weight to imperial units, converting age to years, and creating dummy variables.

For example, the commented-out code includes:

```
# t$height_in <- t$height / 2.54
# t$weight_lbs <- t$weight * 2.20462
# t$age_years <- t$age / 365
```

This suggests that the file `filtered_cardio.csv` already contains the cleaned and transformed variables needed for modeling, such as `age_years`, `height_imperial`, `weight_imperial`, and dummy variables for cholesterol, glucose, and gender.

4 The First Logistic Regression Model: `cardiomodel`

The first main model is:

```
cardiomodel <- glm(cardio ~ age_years + height_imperial + weight_imperial +
  ap_hi + ap_lo + gender_dv +
  cholesterol_2_dv + cholesterol_3_dv +
  gluc_2_dv + gluc_3_dv +
  smoke + alco + active,
  data = t, family = binomial)
summary(cardiomodel)
```

Because `family = binomial` is used, this is a logistic regression model. The model estimates the probability that a patient has cardiovascular disease:

$$p_i = P(\text{cardio}_i = 1).$$

The logistic regression model has the form

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{height}_i + \beta_3 \text{weight}_i + \beta_4 \text{ap_hi}_i + \cdots + \beta_k \text{active}_i.$$

The left-hand side is the log-odds of cardiovascular disease. If we define

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik},$$

then the predicted probability is

$$\hat{p}_i = \frac{1}{1 + e^{-\eta_i}}.$$

This first model includes age, height, weight, systolic blood pressure, diastolic blood pressure, gender, cholesterol dummy variables, glucose dummy variables, smoking, alcohol use, and activity.

5 The Reduced Main-Effects Model: `cardiomodel2`

The second model is a reduced model:

```
cardiomodel2 <- glm(cardio ~ age_years + height_imperial + weight_imperial +
  ap_hi + ap_lo +
  cholesterol_2_dv + cholesterol_3_dv +
  gluc_3_dv +
  smoke + alco + active,
  data = t, family = binomial)
summary(cardiomodel2)
```

This model removes `gender_dv` and `gluc_2_dv`. It keeps only a smaller set of predictors that were considered more important for comparison.

The purpose of this model is to create a simpler and more interpretable logistic regression model. It still models

$$P(\text{cardio} = 1),$$

but with fewer predictors.

6 Comparing `cardiomodel` and `cardiomodel2`

The script compares the full model and the reduced model using AIC and a likelihood ratio test:

```
AIC(cardiomodel, cardiomodel2)
anova(cardiomodel2, cardiomodel, test = "Chisq")
drop1(cardiomodel2, test = "Chisq")
```

6.1 AIC

AIC is the Akaike Information Criterion:

$$AIC = 2k - 2\log(L),$$

where k is the number of estimated model parameters and L is the model likelihood. Lower AIC indicates a better balance between model fit and model simplicity.

6.2 Likelihood Ratio Test

The command

```
anova(cardiomodel2, cardiomodel, test = "Chisq")
```

compares the reduced model to the full model. The test statistic is

$$G^2 = 2\{\log(L_{full}) - \log(L_{reduced})\}.$$

Under the null hypothesis, this approximately follows a chi-square distribution:

$$G^2 \sim \chi_{df}^2.$$

The null hypothesis is that the extra variables in the full model do not significantly improve model fit.

6.3 Single-Term Drop Tests

The command

```
drop1(cardiomodel2, test = "Chisq")
```

tests whether each term in `cardiomodel2` can be removed one at a time. A small p-value means removing that term would significantly worsen the model.

7 DHARMA Residual Diagnostics for cardiomodel2

The script then creates simulated residual diagnostics:

```
sim_res_model_cm2 <- simulateResiduals(cardiomodel2)
testoutliers_cm2 <- testOutliers(simulationOutput = sim_res_model_cm2, type = "bootstrap")

plotQQunif(sim_res_model_cm2)
plotResiduals(sim_res_model_cm2, smoothScatter = FALSE)

testDispersion(sim_res_model_cm2)
testUniformity(sim_res_model_cm2)
```

For logistic regression, ordinary residuals can be difficult to interpret because the response is binary. DHARMA simulates response values from the fitted model and compares the observed data to simulated data. If the model fits well, the DHARMA residuals should be approximately uniformly distributed between 0 and 1.

The diagnostics check:

- Whether the residuals are uniformly distributed.
- Whether there are too many outliers.
- Whether there is overdispersion or underdispersion.
- Whether residual patterns appear across predicted values.

8 Train-Test Prediction Using `cardiomodel2`

The script then performs a 70/30 train-test split:

```
set.seed(123)
n <- nrow(t)
train_idx <- sample(seq_len(n), size = round(0.7 * n))

train <- t[train_idx, ]
test  <- t[-train_idx, ]
```

The model is trained on 70% of the observations and tested on the remaining 30%:

```
cardiomodel2_train <- glm(
  cardio ~ age_years + height_imperial + weight_imperial +
    ap_hi + ap_lo +
    cholesterol_2_dv + cholesterol_3_dv +
    gluc_3_dv +
    smoke + alco + active,
  data = train,
  family = binomial
)
```

Predicted probabilities are computed on the test set:

```
pred_prob <- predict(cardiomodel2_train, newdata = test, type = "response")
```

The output `pred_prob` contains values between 0 and 1:

$$\hat{p}_i = P(\text{cardio}_i = 1 \mid x_i).$$

Using a cutoff of 0.5, the script classifies patients:

```
pred_class <- ifelse(pred_prob >= 0.5, 1, 0)
cm <- table(Predicted = pred_class, Actual = test$cardio)
```

9 Classification Metrics

From the confusion matrix, the script extracts:

- TP : true positives.
- TN : true negatives.
- FP : false positives.
- FN : false negatives.

The metrics are:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

$$Sensitivity = \frac{TP}{TP + FN},$$

$$Specificity = \frac{TN}{TN + FP},$$

$$Precision = \frac{TP}{TP + FP}.$$

Sensitivity is also called recall or the true positive rate. Specificity is the true negative rate. Precision is the proportion of predicted disease cases that were actually disease cases.

10 Density Plot of Predicted Probabilities

The script creates a density plot comparing predicted probabilities for the two actual classes:

```
plot_data <- data.frame(  
  pred_prob = pred_prob,  
  cardio = factor(test$cardio, levels = c(0, 1), labels = c("No Disease", "Disease"))  
)  
  
ggplot(plot_data, aes(x = pred_prob, fill = cardio, color = cardio)) +  
  geom_density(alpha = 0.3) +  
  labs(  
    title = "Density of Predicted Cardiovascular Disease Probabilities",  
    x = "Predicted Probability",  
    y = "Density",  
    fill = "Actual Class",  
    color = "Actual Class"  
  ) +  
  theme_minimal()
```

This graph shows how well the model separates patients with disease from patients without disease. A better model would show the disease group shifted toward higher predicted probabilities and the no-disease group shifted toward lower predicted probabilities.

11 Threshold Tuning

The script evaluates thresholds from 0.01 to 0.99:

```
thresholds <- seq(0.01, 0.99, by = 0.01)
```

For each threshold, the script recalculates accuracy, sensitivity, specificity, precision, Youden's J , F1 score, and the gap between sensitivity and precision.

Youden's statistic is

$$J = \text{Sensitivity} + \text{Specificity} - 1.$$

The F1 score is

$$F1 = \frac{2(\text{Precision})(\text{Sensitivity})}{\text{Precision} + \text{Sensitivity}}.$$

The script identifies three possible thresholds:

- The threshold that maximizes Youden's J .
- The threshold that maximizes the F1 score.
- The threshold where sensitivity and precision are closest.

This is useful because the default cutoff 0.5 may not be the best cutoff for a medical classification problem.

12 Attempted Full Interaction Model: `cardiomodel3`

The script attempts a very large interaction model:

```
cardiomodel3 <- glm(formula = cardio ~ (1 + age_years + height_imperial + weight_imperial + ap_hi + ap_lo +  
  (1 + gender_dv) *  
  (1 + smoke_dv) *  
  (1 + alco_dv) *  
  (1 + active_dv) *  
  (1 + cholesterol_2_dv + cholesterol_3_dv) *  
  (1 + gluc_2_dv + gluc_3_dv), data = t, family = binomial)
```

This formula includes all main effects and many higher-order interactions. The `*` operator in R expands to main effects and interactions. For example,

$$(a + b)^2$$

in formula notation does not mean squaring variables; it means including main effects and pairwise interactions.

This model did not converge. This is expected because the model is extremely large and contains many high-order interactions. Such a model can suffer from overfitting, separation, collinearity, and convergence problems.

13 Forward Selection Attempts: `cardiomodel6`, `cardiomodel6b`, and `cardiomodel6c`

The script then tries to add terms to `cardiomodel2` using likelihood ratio tests.

13.1 Interaction-Block Forward Selection

The first attempt uses:

```
model_try <- update(cardiomodel6, as.formula(paste("~ . *", term)))
```

This adds the candidate term and its interactions with all existing model terms. For each candidate, the script compares the current model and candidate model using a chi-square likelihood ratio test:

```
a <- anova(cardiomodel6, model_try, test = "Chisq")
```

The candidate with the smallest significant p-value is added. This process repeats until no candidate improves the model at $\alpha = 0.05$. However, this approach can be slow because each step may add many interaction terms.

13.2 Main-Effect Forward Selection

The second attempt uses:

```
model_try <- update(cardiomodel6b, as.formula(paste("~ . +", term)))
```

This adds only the candidate main effect instead of all interactions. However, because many of the candidate variables are already included in `cardiomodel2`, this attempt mostly leaves the model unchanged.

13.3 Selected Candidate Forward Selection

The third attempt tests a smaller set of candidate main effects and interactions, such as:

```
"cholesterol_3_dv:gluc_3_dv"  
"age_years:active"  
"weight_imperial:active"  
"ap_hi:ap_lo"
```

This is a more targeted approach, but the script comments that the resulting model remains effectively equal to `cardiomodel2`.

13.4 Important Coding Note

In the loops for `cardiomodel6b` and `cardiomodel6c`, the code assigns the updated model to `cardiomodel6` rather than `cardiomodel6b` or `cardiomodel6c`:

```
cardiomodel6 <- models[[best_term]]
```

If the intention was to update `cardiomodel6b` or `cardiomodel6c`, this line should be changed accordingly. Otherwise, `summary(cardiomodel6b)` and `summary(cardiomodel6c)` may still show the unchanged base model.

14 LASSO Logistic Regression with Pairwise Interactions

Because the full interaction model is too large, the script uses LASSO regression:

```
x <- model.matrix(
  ~ (age_years + height_imperial + weight_imperial +
      ap_hi + ap_lo +
      gender_dv +
      cholesterol_2_dv + cholesterol_3_dv +
      gluc_2_dv + gluc_3_dv +
      smoke + alco + active)^2,
  data = t
)[, -1]
```

The formula includes all main effects and all pairwise interactions, but not higher-order interactions. If there are 13 predictors, then the number of pairwise interactions is

$$\binom{13}{2} = \frac{13(12)}{2} = 78.$$

Thus the model matrix contains roughly

$$13 + 78 = 91$$

predictor columns.

LASSO logistic regression solves the penalized optimization problem

$$\min_{\beta} \left[-\log L(\beta) + \lambda \sum_{j=1}^p |\beta_j| \right].$$

The penalty term $\lambda \sum |\beta_j|$ shrinks some coefficients exactly to zero. Terms with zero coefficients are removed from the model.

The cross-validated LASSO model is fit using:

```
cardiolarasso_cv <- cv.glmnet(
  x, y,
  family = "binomial",
  alpha = 1,
  nfolds = 10,
  standardize = TRUE
)
```

Here, `alpha = 1` specifies LASSO. The script uses 10-fold cross-validation to choose λ . It then uses `lambda.1se`, which chooses a simpler model within one standard error of the best cross-validation error:

```
coef_1se <- coef(cardiolarasso_cv, s = "lambda.1se")
```

15 Selected Terms and the Hierarchy Principle

The script extracts nonzero LASSO coefficients:

```
selected_terms <- rownames(coef_1se)[as.vector(coef_1se !=0)]
selected_terms <- setdiff(selected_terms, "(Intercept)")
```

If LASSO selects an interaction term, the script also includes its parent main effects:

```
interaction_terms <- selected_terms[grepl(":", selected_terms)]

if (length(interaction_terms) > 0) {
  main_effects_interactions <- unique(unlist(strsplit(interaction_terms, ":")))
  selected_terms <- unique(c(selected_terms, main_effects_interactions))
}
```

This follows the hierarchy principle. For example, if the model includes

$$ap_hi : cholesterol_3_dv,$$

it should also include

$$ap_hi \text{ and } cholesterol_3_dv.$$

This makes the interaction model easier to interpret.

16 Post-LASSO Logistic Regression: `cardiomodel8`

After LASSO selects terms, the script refits those terms with ordinary logistic regression:

```
lasso_formula <- as.formula(
  paste("cardio ~", paste(selected_terms, collapse = " + "))
)

cardiomodel8 <- glm(lasso_formula, data = t, family = binomial)
```

This is called a post-LASSO model. LASSO is used for variable selection, and then ordinary logistic regression is used to estimate coefficients and p-values without the LASSO shrinkage penalty.

17 Pruning the Post-LASSO Model

The script then prunes `cardiomodel8` by keeping:

- significant interaction terms with p-values below 0.05,
- significant main effects with p-values below 0.05,
- parent main effects for any kept interaction.

This creates `cardiomodel8_reduced`:

```
cardiomodel8_reduced <- glm(
  cardiomodel8_reduced_formula,
  data = t,
  family = binomial
)
```

The final model is compared to the original post-LASSO model using AIC and a likelihood ratio test:

```
AIC(cardiomodel2, cardiomodel8_reduced, cardiomodel8)
anova(cardiomodel8_reduced, cardiomodel8, test = "Chisq")
drop1(cardiomodel8_reduced, test = "Chisq")
```

The goal is to see whether the reduced post-LASSO model keeps most of the predictive value while remaining simpler than the full post-LASSO model.

18 Residual Diagnostics for the LASSO-Based Models

The script performs DHARMa diagnostics for both `cardiomodel8` and `cardiomodel8_reduced`:

```
sim_res_model_cm8 <- simulateResiduals(cardiomodel8)
plotQQunif(sim_res_model_cm8)
plotResiduals(sim_res_model_cm8, smoothScatter = FALSE)
testDispersion(sim_res_model_cm8)
testUniformity(sim_res_model_cm8)

sim_res_model_cm8_r <- simulateResiduals(cardiomodel8_reduced)
plotQQunif(sim_res_model_cm8_r)
plotResiduals(sim_res_model_cm8_r, smoothScatter = FALSE)
testDispersion(sim_res_model_cm8_r)
testUniformity(sim_res_model_cm8_r)
```

These diagnostics check whether the final logistic models show evidence of poor fit, outliers, non-uniform residuals, or dispersion problems.

19 ROC Curve and AUC

The script evaluates the reduced post-LASSO model using an ROC curve:

```
probs <- as.numeric(unlist(predict(cardiomodel8_reduced, type = "response")))
actuals <- as.numeric(unlist(t$cardio))
roc_obj <- roc(actuals, probs)
auc(roc_obj)
plot(roc_obj, col= "lightblue", lwd=3, main = "ROC Curve for cm8_reduced")
```

The ROC curve plots sensitivity against false positive rate across all possible thresholds. The false positive rate is

$$FPR = \frac{FP}{FP + TN}.$$

The AUC measures how well the model separates patients with cardiovascular disease from patients without cardiovascular disease. A common interpretation is:

- AUC = 0.50: random guessing.
- AUC around 0.70: acceptable separation.
- AUC around 0.80 or higher: strong separation.
- AUC = 1.00: perfect separation.

20 Odds Ratio Plot for Presentation

The script creates an odds ratio plot for `cardiomodel2`:

```
or_table <- data.frame(
  term = names(coef(cardiomodel2)),
  OR = exp(coef(cardiomodel2)),
  lower = exp(confint(cardiomodel2)[,1]),
  upper = exp(confint(cardiomodel2)[,2])
)
```

For a logistic regression coefficient β_j , the odds ratio is

$$OR_j = e^{\beta_j}.$$

Interpretation:

- If $OR_j > 1$, the predictor increases the odds of cardiovascular disease.
- If $OR_j < 1$, the predictor decreases the odds of cardiovascular disease.
- If $OR_j = 1$, the predictor has no association with the odds of disease.

The 95% confidence interval is created by exponentiating the coefficient confidence interval:

$$e^{\hat{\beta}_j \pm 1.96SE(\hat{\beta}_j)}.$$

The plot includes a red dashed line at $OR = 1$. Terms with confidence intervals entirely above or below 1 are easier to interpret as statistically significant risk-increasing or risk-decreasing predictors.

21 AIC Comparison Plot

The script creates a bar plot comparing AIC values for:

- the main-effects model `cardiomodel2`,
- the full post-LASSO model `cardiomodel8`,
- the reduced post-LASSO model `cardiomodel8_reduced`.

The model with the lowest AIC has the best balance of goodness of fit and simplicity among the compared models.

22 Important Interactions Identified

The script comments that the reduced post-LASSO model selected several important interactions, including:

- `age_years:ap_hi`,
- `ap_hi:cholesterol_3_dv`,

- `weight_imperial:ap_hi`,
- `cholesterol_3_dv:gluc_3_dv`.

For example, the interaction `age_years:ap_hi` means the effect of systolic blood pressure on cardiovascular disease risk may depend on age. A model containing this interaction has the form

$$\eta_i = \cdots + \beta_1 age_i + \beta_2 ap_hi_i + \beta_3 (age_i \cdot ap_hi_i).$$

Then the effect of systolic blood pressure on the log-odds of disease is

$$\frac{\partial \eta_i}{\partial ap_hi_i} = \beta_2 + \beta_3 age_i.$$

Thus, if $\beta_3 \neq 0$, the effect of systolic blood pressure changes with age.

The interaction `cholesterol_3_dv:gluc_3_dv` means the combined effect of well-above-normal cholesterol and well-above-normal glucose may differ from the sum of their separate effects.

23 Overall Interpretation of the Script

Overall, `diego_analysis.R` performs a complete logistic regression workflow for cardiovascular disease risk:

1. Load the cleaned cardiovascular disease dataset.
2. Fit a full main-effects logistic regression model.
3. Fit a smaller, more interpretable main-effects logistic regression model.
4. Compare models using AIC and likelihood ratio tests.
5. Check residual diagnostics using DHARMA.
6. Split the data into training and testing sets.
7. Compute predicted probabilities and classification metrics.
8. Tune the probability threshold using Youden's statistic, F1 score, and sensitivity-precision balance.
9. Attempt a very large interaction model, which does not converge.
10. Use LASSO logistic regression to search through pairwise interactions without making the model explode.
11. Refit LASSO-selected terms using ordinary logistic regression.
12. Prune the post-LASSO model while preserving interaction hierarchy.
13. Evaluate final models using AIC, DHARMA diagnostics, ROC curves, and AUC.
14. Create odds ratio and AIC plots for presentation.

24 Recommended Summary for a Presentation

A concise explanation of the file is:

This script models cardiovascular disease status using logistic regression. It first fits and evaluates a main-effects model using age, body measurements, blood pressure, cholesterol, glucose, smoking, alcohol use, and activity. Model quality is assessed using AIC, likelihood ratio tests, classification metrics, and DHARMA residual diagnostics. The script then explores more complex interaction models. Since a full interaction model fails to converge, LASSO logistic regression is used to select important pairwise interactions. The selected terms are refit in a regular logistic regression model, pruned while preserving hierarchy, and evaluated using ROC/AUC and diagnostic plots. Odds ratio and AIC plots are produced for final presentation.

25 Potential Improvements

A few improvements could make the script cleaner and easier to reproduce:

- Avoid running `install.packages()` every time the script runs; package installation should usually be separate from analysis.
- Replace the absolute file path with a relative path, such as `read.csv2("filtered_cardio.csv")`.
- Avoid naming the dataset `t`, since `t()` is also a base R transpose function.
- Fix the update line in the `cardiomodel6b` and `cardiomodel6c` loops if those models are intended to update themselves.
- Store model comparison results and classification metrics in clean tables for reporting.
- Use consistent naming for variables and models.